Guía de desarrollo — Iker

Iker Esteban Fuentes Torres · M7 Administración y moderación · M6 Reseñas · Offero, versión 4 · 30 de octubre de 2026 · Wilson responsable de base de datos

> Esta guía es tu hoja de ruta para tus módulos: qué construir, en qué orden, con qué tablas y permisos, y cómo saber que quedó bien. No trae código: el código lo escribes tú. Las historias con puntos van en el Product Backlog del plan (Parte 5). Si algo aquí choca con el documento de contexto, manda el documento y le avisas a Diego. Lo marcado con [SUPUESTO] es una propuesta que todavía hay que confirmar.

Tu resumen

Módulos	M7 Administración (complejidad media-alta: ≈30 h completo, ≈16 h mínimo) y M6 Reseñas (media-baja: ≈18 h completo, ≈10 h mínimo).
Carga estimada	≈69 h planificadas frente a ≈76 h disponibles hasta el 29 de enero, incluido tu rol de revisor de seguridad (RLS) y «Mis cotizaciones» (OFF-031).
Orden de trabajo	M7 primero (Sprints 1 a 4), porque M6 depende de que existan cotizaciones aceptadas (M4, núcleo del dom 6 dic).
Prioridad	M7 mínimo termina el dom 6 dic y M6 mínimo el vie 15 ene. Los extras de M7 (cambiar rol, suspender) y M6 completo van en enero. Si hay que recortar, se recortan extras de M7, no M6.
Desbloqueas a	Todo el equipo (botón de reportar y moderación) y Daniel (componentes de reseñas para su página).
Dependes de	Diego (M1 núcleo, mar 10 nov; función de aceptar, vie 4 dic), Daniel (categories, businesses) y Wilson (requests).
Fechas clave	/administrador mar 10 nov · reports y ReportButton dom 15 nov · categorías y usuarios dom 22 nov · reportes en panel y «Mis cotizaciones» dom 29 nov · **M7 mínimo dom 6 dic** · reviews en dev dom 13 dic · formulario de reseña vie 18 dic · **M6 mínimo vie 15 ene** · M6 y M7 completos vie 22 ene.

Tu calendario por sprint

Sprint	Qué entregas
0 · 28 oct – 8 nov	Preparar tu computador; pareja con Diego en profiles y funciones (lun 2 – mar 3); práctica de RLS en dev; checklist de RLS; diseño de la BD (jue 5).
1 · 9 – 15 nov	Proteger /administrador (mar 10); pareja con Wilson en requests; reports + ReportButton (dom 15); revisión de RLS de las primeras tablas.
2 · 16 – 22 nov	Prueba cruzada de M1 (mié 18); categorías y lista de usuarios en el panel (dom 22).
3 · 23 – 29 nov	Reportes en el panel; «Mis cotizaciones» del proveedor (OFF-031, apoyando a Diego) (dom 29).
4 · 30 nov – 6 dic	Revisar la función de aceptar; ocultar contenido: **M7 mínimo terminado (dom 6 dic)**.
5 · 7 – 13 dic	Semana de finales (media carga): tabla reviews + RLS + promedio en dev (dom 13).
6 · 14 – 18 dic	Formulario de reseña (vie 18).
Descanso	19 dic – 3 ene.
7 · 4 – 15 ene	Promedio y lista con Daniel: **M6 mínimo (vie 15 ene)**; cambiar roles con Diego.
8 · 18 – 29 ene	Suspender cuentas y editar/ordenar reseñas (hasta vie 22); revisión final de seguridad.

Ruta de aprendizaje (justo a tiempo)

Aprende cada tema justo antes de la tarea que lo necesita, no todo al principio. El orden importa: cada punto se apoya en el anterior.

Sprint 0

1. Git en equipo (Anexo C).
2. JavaScript moderno: const y let, funciones flecha, map y filter, desestructuración, async/await.
3. React y Next.js App Router.
4. SQL básico con JOIN.
5. Qué es RLS y cómo se prueba con dos cuentas de roles distintos.

Antes de M7 (Sprints 1 y 2)

1. Proteger rutas por rol con el patrón de Diego.
2. Server Actions con validación en el servidor.
3. Políticas para admin con `is_admin()`.
4. Tablas que apuntan a «cualquier cosa» (`target_type` + `target_id`) y por qué no llevan clave foránea.

Antes de M6 (Sprint 5)

1. Restricciones UNIQUE y CHECK.
2. RLS con condiciones sobre otras tablas (cotizaciones aceptadas).
3. Consultas de agregación (promedio y conteo) o vistas.

Tu rol transversal: revisor de seguridad (RLS)

- En cada PR que cree o cambie una tabla, revisa la checklist: RLS activada; políticas para SELECT, INSERT, UPDATE y DELETE; nada abierto a todos sin razón; la clave secreta nunca en el frontend.
- En la prueba cruzada de cada módulo, intenta «romper» los permisos con una cuenta de otro rol (por ejemplo, cambiando ids en la URL).
- Revisa con especial cuidado el trigger de perfiles y la función de aceptar de Diego: son funciones con privilegios elevados.
- **Wilson es el responsable de la base de datos** (esquema y SQL en prod); tú cuidas que los permisos sean correctos. Un PR con SQL necesita la revisión de los dos.

Módulo 7 — Panel de administración y moderación

Objetivo	Panel /administrador, solo para admins, para gestionar categorías, consultar usuarios, recibir y resolver reportes, y ocultar o mostrar contenido.
Alcance	**Mínimo (hasta el dom 6 dic):** proteger el panel, categorías, lista de usuarios, reportes (crear y revisar) y ocultar contenido. **Completo (enero):** cambiar el rol de un usuario y suspender cuentas.
Fuera de alcance	Borrar cuentas, métricas avanzadas, historial completo de moderación y moderar mensajes privados.
Depende de	Diego (M1); Daniel (categories, businesses); Wilson (requests); tus propias reseñas (M6) para moderarlas.
Desbloquea a	Botón «Reportar» para todo el equipo.

Tabla `reports`

Campo	Tipo sugerido	Reglas
id	uuid	Clave primaria, automática.
reporter_id	uuid → profiles.id	Quien reporta: el usuario actual.
target_type	text	business, request o review (CHECK) [SUPUESTO: los mensajes privados quedan fuera del MVP].
target_id	uuid	Id del elemento reportado. Sin clave foránea, porque puede ser de tres tablas distintas.
reason	text	Obligatorio, 10 a 500 caracteres.
status	text	open, reviewed o dismissed; por defecto open.
created_at	timestamptz	Automático.
Única (reporter_id, target_type, target_id)	—	Un usuario reporta el mismo elemento una sola vez.

Políticas RLS de `reports`

Operación	Quién	Condición
SELECT	Usuario, admin	El que reportó ve los suyos; el admin ve todos.
INSERT	Usuario con sesión	`reporter_id` es el usuario actual; status open.
UPDATE	Admin	Cambiar status.
DELETE	Nadie	—

Políticas de admin en tablas de otros

Van en tu propio archivo SQL y las revisa el dueño de cada tabla antes de fusionar.

Tabla	Dueño	Permiso del admin
`categories`	Daniel	Crear, renombrar y borrar (acuerden quién escribe la política).
`businesses`	Daniel	Cambiar `is_hidden`.
`requests`	Wilson	Cambiar `is_hidden`.
`reviews`	Iker	Cambiar `is_hidden`.
`profiles`	Diego	Cambiar `role` (extra).

Borrar una categoría en uso debe fallar, porque la clave foránea no tiene borrado en cascada. Muestra un mensaje claro: «Categoría en uso; renómbrala en vez de borrarla».

Rutas y archivos

- `src/app/administrador/page.js` — resumen: reportes abiertos y conteos. Protegido en el servidor con `is_admin()`; además las políticas bloquean todo aunque alguien llame a Supabase directo.
- `src/app/administrador/categorias/page.js`, `reportes/page.js` y `usuarios/page.js` [SUPUESTO: subrutas nuevas].
- `src/components/ui/ReportButton.js` — botón «Reportar» reutilizable: Daniel lo pone en negocios, Wilson en solicitudes y tú en reseñas.
- `src/services/admin.js` y `src/services/reports.js`.

Pasos en orden (cada paso es un pull request pequeño)

#	Paso	Fecha meta	Cómo verificar que quedó bien
1	Proteger /administrador con el patrón de Diego.	mar 10 nov	Una cuenta no admin es redirigida; la de Diego (admin) entra.
2	Tabla reports + RLS + ReportButton + Server Action.	dom 15 nov	Un reporte repetido no se duplica.
3	Categorías: lista, crear, renombrar y borrar si no está en uso (con Daniel).	jue 19 nov	Una categoría nueva aparece en el filtro de negocios.
4	Lista de usuarios: nombre, rol y fecha de registro.	dom 22 nov	Solo el admin ve la lista.
5	Reportes: lista, ver el elemento reportado, marcar revisado o descartado.	sáb 28 nov	El estado cambia y queda registrado.
6	Ocultar o mostrar negocios y solicitudes (políticas de admin en tablas ajenas). Reseñas: cuando exista su tabla (dic).	jue 3 dic	Un negocio oculto desaparece del listado público.
7	Prueba cruzada (la hace Diego) y publicación. **M7 mínimo terminado.**	dom 6 dic	Checklist de la Definition of Done completo (Anexo G).
8	Cambiar el rol de un usuario (política de admin sobre profiles, con Diego).	vie 15 ene	Un admin cambia un cliente a proveedor; nadie más puede.
9	Suspender cuentas (campo is_suspended acordado con Diego) + prueba cruzada. **M7 completo.**	vie 22 ene	Una cuenta suspendida no puede iniciar sesión ni crear contenido.

Criterios de aceptación

- **Dado** un usuario que no es admin, **cuando** abre /administrador o llama directo a una operación de admin, **entonces** es redirigido o la operación falla.
- **Dado** un admin, **cuando** crea una categoría, **entonces** aparece en el filtro de negocios y en el formulario de solicitudes.
- **Dado** un admin, **cuando** intenta borrar una categoría en uso, **entonces** el sistema lo impide y explica por qué.
- **Dado** un usuario, **cuando** reporta una reseña con un motivo, **entonces** el reporte aparece como abierto en el panel; si reporta lo mismo otra vez, no se duplica.
- **Dado** un admin, **cuando** oculta un negocio reportado, **entonces** deja de aparecer en los listados públicos, pero el dueño y el admin todavía lo ven.

Errores típicos y cómo evitarlos

- Proteger el panel solo ocultando el enlace del menú.
- Escribir políticas de admin en tablas ajenas sin avisar al dueño de la tabla.
- Mostrar correos de usuarios fuera del panel de admin.
- Borrar contenido en vez de ocultarlo: se pierde el historial y se rompen relaciones.

Coordinación con el equipo

- Daniel: acuerden quién escribe la política de admin en categories; él coloca ReportButton en el detalle del negocio.
- Wilson: coloca ReportButton en el detalle de la solicitud.
- Diego: hace la prueba cruzada de M7 y aprueba tus políticas sobre tablas ajenas.

Módulo 6 — Reseñas y valoraciones

Objetivo	El cliente que aceptó una cotización califica al negocio de 1 a 5 con un comentario, una vez por solicitud; la página del negocio muestra el promedio y las reseñas.
Alcance	**Mínimo (hasta el vie 15 ene):** crear la reseña, mostrar promedio y lista, ocultar desde el admin. **Completo (hasta el vie 22 ene):** el autor edita su reseña durante 7 días [SUPUESTO] y la lista se ordena por fecha o puntuación.
Fuera de alcance	Respuesta del negocio a la reseña, fotos y reseñas de clientes hechas por proveedores.
Depende de	Diego (M4: cotizaciones aceptadas, vie 4 dic); Wilson (detalle de la solicitud, donde va el formulario); Daniel (detalle del negocio, donde se muestran las reseñas).
Desbloquea a	Nadie: es un módulo final.

Regla contra reseñas falsas [SUPUESTO; decisión pendiente de la sección 16]: solo reseña el cliente de una solicitud adjudicada, al negocio cuya cotización fue aceptada, una sola vez por solicitud. Opción más estricta para debatir: exigir que el cliente marque «trabajo completado» (requiere un estado nuevo en requests; fuera del MVP).

Tabla `reviews`

Campo	Tipo sugerido	Reglas
id	uuid	Clave primaria, automática.
reviewer_id	uuid → profiles.id	El cliente que reseña: el usuario actual.
business_id	uuid → businesses.id	El negocio reseñado.
request_id	uuid → requests.id	Única: una reseña por solicitud.
rating	integer	Entre 1 y 5 (CHECK).
comment	text	Obligatorio, 10 a 1000 caracteres [SUPUESTO].
is_hidden	boolean	Falso por defecto; solo el admin lo cambia.
created_at	timestamptz	Automático.

Políticas RLS de `reviews`

Operación	Quién	Condición
SELECT	Todos	Las no ocultas; el autor y el admin ven también las ocultas.
INSERT	Cliente	`reviewer_id` es el usuario actual; es el cliente de `request_id`; existe una cotización accepted de `business_id` para esa solicitud.
UPDATE	Admin (mínimo)	Solo `is_hidden`. En la versión completa, el autor edita rating y comment durante 7 días.
DELETE	Nadie	—

Promedio: una vista (por ejemplo `business_ratings`, con business_id, promedio y cantidad) o una consulta de agregación en tu servicio. Debe excluir las reseñas ocultas. Si usas vista, revisa con Diego que no exponga datos de más.

Rutas y archivos

- `src/components/reviews/ReviewForm.js` — Wilson lo coloca en el detalle de la solicitud cuando está adjudicada, el usuario es el dueño y aún no reseñó.
- `src/components/reviews/RatingSummary.js` y `ReviewList.js` — Daniel los coloca en el detalle del negocio; cada reseña lleva ReportButton.
- `src/services/reviews.js`.

Pasos en orden (cada paso es un pull request pequeño)

#	Paso	Fecha meta	Cómo verificar que quedó bien
1	Tabla reviews + RLS + vista de promedio en dev (usa cotizaciones aceptadas de prueba en dev).	dom 13 dic	Un usuario sin cotización aceptada no puede insertar.
2	ReviewForm + Server Action con validaciones; Wilson lo coloca en la solicitud adjudicada.	vie 18 dic	Una segunda reseña de la misma solicitud se rechaza.
3	RatingSummary y ReviewList; integración con Daniel.	vie 8 ene	El promedio coincide con un cálculo hecho a mano.
4	Prueba cruzada (la hace Wilson) y publicación con una reseña real del piloto. **M6 mínimo terminado.**	vie 15 ene	Checklist de la Definition of Done completo (Anexo G).
5	Editar reseña durante 7 días y ordenar la lista. **M6 completo.**	vie 22 ene	Después de 7 días ya no se puede editar; el orden por puntuación es correcto.

Ojo con la regla de «sin datos ficticios»: para marcar M6 como Terminado necesitas al menos una reseña real, y eso exige que una empresa del piloto haya aceptado una cotización real. Coordínalo con Diego (PO) desde el núcleo (6 dic).

Criterios de aceptación

- **Dado** un cliente cuya solicitud se adjudicó al negocio A, **cuando** deja 4 estrellas y un comentario, **entonces** la reseña aparece en la página de A y el promedio se actualiza.
- **Dado** el mismo cliente, **cuando** intenta reseñar otra vez la misma solicitud, **entonces** el sistema lo impide.
- **Dado** un cliente con la solicitud todavía abierta, o un usuario que nunca contrató a A, **cuando** intenta reseñar a A desde la app o directo contra Supabase, **entonces** la operación falla.
- **Dado** un admin, **cuando** oculta una reseña, **entonces** deja de mostrarse y de contar en el promedio.

Errores típicos y cómo evitarlos

- Validar la regla de reseñas solo en el formulario.
- Un promedio que incluye reseñas ocultas.
- Guardar la puntuación como texto.

Coordinación con el equipo

- Diego: avísale cuando tengas la tabla; necesitas su función de aceptar (vie 4 dic) para probar con datos reales.
- Wilson: coloca ReviewForm en el detalle de la solicitud adjudicada.
- Daniel: coloca RatingSummary y ReviewList en el detalle del negocio.

Anexo — Acuerdos comunes del equipo

Este anexo es igual en las cuatro guías. Si cambia, se cambia en todas.

A. Calendario del equipo

Sprint	Fechas	Meta del equipo
0	mié 28 oct – dom 8 nov	Primer despliegue dom 1 nov; M0 terminado sáb 7 nov; entornos y tablero listos; profiles y funciones de ayuda en dev
1	lun 9 – dom 15 nov	**M1 núcleo mar 10 nov**; registro e inicio de sesión; tablas categories, businesses, requests y reports con RLS
2	lun 16 – dom 22 nov	M1 terminado (mié 18); quotes en dev; «Mi negocio»; formulario de solicitud; categorías y usuarios en el panel
3	lun 23 – dom 29 nov	Primera cotización enviada; listados y detalle de negocios y solicitudes; reportes en el panel
4	lun 30 nov – dom 6 dic	**Núcleo completo (dom 6 dic):** solicitud → cotización → elegir con empresas reales; M2, M3, M4 y M7 mínimo terminados
5	lun 7 – dom 13 dic	Semana de finales: media carga [SUPUESTO]. Estabilizar el núcleo; messages y reviews en dev
6	lun 14 – vie 18 dic	Enviar mensajes y formulario de reseña; uso real del núcleo; avance con el profesor
—	sáb 19 dic – dom 3 ene	**Descanso: nadie trabaja**
7	lun 4 – vie 15 ene (2 semanas)	**MVP v1 (vie 15 ene):** los 8 módulos publicados en versión mínima
8	lun 18 – vie 29 ene (2 semanas)	Versiones completas hasta el **congelamiento vie 22 ene**; estabilización y presentación. **Entrega: vie 29 ene** [SUPUESTO]

Festivos en Colombia en días de trabajo: lun 2 nov, lun 16 nov, mar 8 dic y lun 11 ene. Sin subir horas: cada uno mantiene sus 12 h semanales (Diego 14 h), también en enero [SUPUESTO].

B. Preparar tu computador (Windows, PowerShell)

#	Paso	Comando o acción	Cómo verificar
1	Instalar Node.js LTS	`winget install OpenJS.NodeJS.LTS` (o el instalador LTS de nodejs.org)	`node -v` y `npm -v` muestran versión
2	Instalar Git	`winget install Git.Git`	`git --version`
3	Permitir scripts de npm	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (responde S)	`npx -v` funciona sin el error «scripts deshabilitados»

4	Configurar Git	`git config --global user.name "Tu Nombre"`
		`git config --global user.email "correo de tu GitHub"`
		`git config --global init.defaultBranch main`
			`git config --global --list`

5	Extensiones de VS Code	ESLint, Tailwind CSS IntelliSense y Prettier; opcional: GitLens	Aparecen en la pestaña Extensiones

6	Clonar fuera de OneDrive	`mkdir C:\dev`
		`cd C:\dev`
		`git clone https://github.com/diegodani2030/App-Offero.git`
			Existe `C:\dev\App-Offero`

7	Instalar dependencias (cuando M0 esté en main)	`cd C:\dev\App-Offero` `npm install`	Termina sin errores en rojo

#	Paso	Comando o acción	Cómo verificar
8	Variables de entorno	`Copy-Item .env.example .env.local` y llena los valores que Diego te envía por mensaje privado	`.env.local` existe y `git status` no lo muestra
9	Correr la app	`npm run dev` y abre http://localhost:3000	Ves la portada de Offero

Abre una PowerShell nueva después de instalar Node y Git. El correo de Git debe ser el de tu cuenta de GitHub. La primera vez que hagas push, Windows abrirá el navegador para iniciar sesión en GitHub.

C. Flujo de Git paso a paso

#	Qué haces	Comando o acción
1	Toma una tarea del tablero y muévela a «En curso»	GitHub Projects
2	Actualiza main	`git switch main` → `git pull`
3	Crea tu rama: tipo/mN-descripcion (tipos: feat, fix, chore, docs)	`git switch -c feat/m3-formulario-solicitud`
4	Guarda avances pequeños con mensajes en español: tipo(módulo): qué hace	`git add .` → `git commit -m "feat(m3): agrega formulario de nueva solicitud"`
5	Sube la rama	`git push -u origin feat/m3-formulario-solicitud` (después, solo `git push`)
6	Abre el pull request hacia main con la plantilla: qué cambia, cómo probarlo, capturas y checklist. Escribe «Cierra #N» para enlazar la tarea	GitHub
7	Revisión: al menos un compañero. Si trae SQL, revisa Wilson (base de datos) e Iker (RLS); si toca autenticación o archivos compartidos, revisa Diego. Corrige con nuevos commits en la misma rama	GitHub
8	Fusión: la hace Diego con «Create a merge commit» (por la restricción de Vercel Hobby); luego se borra la rama	GitHub
9	Actualiza tu rama con main cada día	`git switch main` → `git pull` → `git switch tu-rama` → `git merge main`
10	Si hay conflicto, VS Code muestra las opciones (cambio actual, entrante o ambos). Si no entiendes el otro cambio, pregúntale a su autor. Luego guarda y haz commit	`git add .` → `git commit`

- Nunca hagas commits directos a main ni `git push --force`.
- Pull requests de menos de un día de trabajo: son más fáciles de revisar y casi no generan conflictos.
- No subas `.env.local`, `node_modules` ni claves. Si `git status` los muestra, detente y pregunta.

D. Convenciones de base de datos

- **Dos proyectos de Supabase** [SUPUESTO; se confirma en la Parte 3]: `offero-dev` para desarrollo y pruebas (ahí sí puedes crear datos de prueba) y `offero-prod` solo con datos reales (lo usa la versión de producción en Vercel). Tu `.env.local` y las previews apuntan a dev.
- **Cada cambio de esquema es un archivo** en `supabase/sql/` con nombre `AAAAMMDD_HHMM_descripcion.sql` (por ejemplo `20261111_1530_requests.sql`). La fecha y hora evitan choques entre cuatro personas. Se prueba en dev, va en el PR, lo revisa Wilson y Wilson (o Diego, de respaldo) lo aplica en prod antes de fusionar. Nunca edites un archivo ya aplicado: crea uno nuevo. Nadie cambia tablas desde el panel sin dejar su archivo.
- **Todas las tablas**: id uuid automático (profiles usa el id de Auth), created_at automático, claves foráneas explícitas y restricciones CHECK para estados y números.

- **RLS se activa en el mismo archivo que crea la tabla.** Una tabla sin políticas no se fusiona.
- **Contenido moderable** (businesses, requests, reviews) incluye `is_hidden` desde su creación, para que M7 no tenga que modificar tablas ajenas [SUPUESTO; registrar en la sección 11 del documento].
- **Funciones de ayuda de Diego**: `is_admin()` y `get_my_role()`. Úsenlas en las políticas en vez de repetir consultas.
- **Nombres de personas**: siempre desde la vista `public_profiles`. Los correos nunca se muestran a otros usuarios.
- **Validación en tres capas**: formulario (comodidad), servidor de Next.js (seguridad) y base de datos (restricciones y RLS, la última barrera).

Tabla.campo	Valores acordados (fuente única)
`profiles.role`	client, provider, admin
`requests.status`	open (recibe cotizaciones), awarded (adjudicada), cancelled
`quotes.status`	pending, accepted, rejected, withdrawn
`reports.status`	open, reviewed, dismissed
`reports.target_type`	business, request, review

E. Archivos compartidos

Archivo o carpeta	Dueño	Regla
`src/app/layout.js`, `globals.css`, `src/components/layout/`	Daniel	Avisa en Discord antes de cambiarlos; cambios pequeños
`src/lib/supabase/` y archivo de sesión (proxy o middleware)	Diego	Nadie crea otro cliente de Supabase
`supabase/sql/`	Dueño de cada tabla; aprueba Wilson	No editar archivos ya aplicados
`package.json` y `package-lock.json`	Todos, con acuerdo	No instales paquetes sin acordarlo (restricción de stack). Si hay conflicto en el lock: acepta la versión de main y vuelve a ejecutar `npm install`
`docs/` (bóveda de Obsidian)	Cada uno, su módulo; coordina Daniel	Actualiza tu nota de módulo y las de tus tablas al avanzar
`src/components/ui/`	Quien crea el componente	Si cambias un componente compartido (ReportButton, CategoryFilter), avisa

F. Comunicación, regla de atasco e IA

- **Canales** [SUPUESTO]: Discord para el trabajo (#daily para el avance diario, #ayuda para bloqueos, #prs para pedir revisión). WhatsApp solo para urgencias.
- **Regla de atasco**: después de 45 minutos intentando solo, escribe en #ayuda qué intentabas, el archivo, el mensaje de error completo y qué probaste. Si en 2 horas nadie responde, menciona a Diego. Un bloqueo de más de un día se dice en la Daily.
- **Revisiones**: responde las que te pidan en menos de 24 horas.
- **IA**: pega el documento de contexto al iniciar cada conversación; pide explicaciones, no solo código; verifica que use JavaScript, App Router y `@supabase/ssr` (no TypeScript, ni la carpeta `pages/`, ni Flask); no subas nada que no puedas explicar en la revisión; nunca pegues claves en la IA.

G. Definition of Done

Una historia está hecha cuando:

- cumple sus criterios de aceptación;
- su pull request fue revisado y fusionado;
- funciona en la Preview y luego en producción;
- sus permisos se probaron con dos cuentas de roles distintos;
- no deja errores en la consola del navegador;
- el documento de contexto refleja cualquier cambio.

Un módulo está Terminado cuando (regla del documento de contexto):

- funciona en la versión publicada en Vercel;
- sus operaciones principales están probadas por otra persona (prueba cruzada);
- no depende de datos ficticios en producción;
- la sección 8 del documento dice Terminado y Publicado: Sí.